TRABAJO PRÁCTICO FINAL • CERTIFICACIÓN ARQUITECTO DE AUTOMATIZACIÓN IA

Construcción de tu Ecosistema de Automatización IA

Sistema de Triage Inteligente de Consultas Comerciales y Soporte vía WhatsApp Cloud API, n8n, Gemini y Airtable CRM

AUTOR DEL PROYECTO
Javier Alejandro Ruiz Hurtado

ROL & ARQUITECTURA
Lead AI Automation Engineer (n8n + LLM)

ENTORNO & FECHA
n8n Self-Hosted • Octubre 2026

1. Elige tu Caso de Uso

Proceso Seleccionado: Triage, Clasificación Semántica y Calificación Automatizada de Consultas de Atención al Cliente y Soporte vía WhatsApp Business Cloud API.

Problema de Negocio: En canales conversacionales masivos, la saturación por consultas repetitivas de baja complejidad (horarios, ubicación, dudas comerciales básicas) degrada los tiempos de atención e incrementa los costos operativos. En simultáneo, las solicitudes de alto valor (presupuestos a medida, quejas críticas, requerimientos personalizados) sufren demoras de derivación, deteriorando la conversión y la satisfacción del usuario.

Solución Implementada: Un ecosistema de automatización end-to-end donde cada interacción entrante es capturada en tiempo real mediante un webhook transaccional, registrada en Airtable (CRM de Contactos y Tickets), analizada cognitivamente por Google Gemini para determinar su criticidad (*GENERAL* vs. *ESPECIFICA*) y enrutada dinámicamente: auto-resolución instantánea para dudas básicas o escalación prioritaria al WhatsApp personal del asesor con síntesis ejecutiva del requerimiento.

2. Estructura el "Cerebro" (Base de Datos Relacional)

La infraestructura de memoria y estado se diseñó en **Airtable CRM** mediante un modelo relacional de dos tablas enlazadas que garantizan la integridad referencial y evitan datos aislados:

Tabla	Campo	Tipo de Dato	Función & Propósito Técnico
Contactos *(Entidad Cliente)*	`Telefono_ID`	Single line text (Key)	Identificador único internacional (formato E.164 ej. +54911...).
	`Nombre_WhatsApp`	Single line text	Nombre de perfil obtenido de la carga útil del webhook.
	`Tickets_Relacionados`	Link to Tickets	Relación relacional 1 a N con la tabla de atenciones.
Tickets *(Gestión de Atenciones)*	`ID_Ticket`	Autonumber (Primary)	Identificador secuencial auditable para cada evento.
	`Contacto_Asociado`	Link to Contactos	Enlace relacional al registro del cliente.
	`Estado`	Single Select	Valores obligatorios: `Pendiente`, `Procesado por IA`, `Escalado a Humano`, `Cerrado`.
	`Categoria_IA`	Single Select	Categoría normalizada: `GENERAL` o `ESPECIFICA`.
	`Resumen_Caso`	Long Text	Síntesis estructurada provista por Gemini para el operador.

> **SINCRONIZACIÓN TRANSACCIONAL**
>
> Para evitar duplicidades en la base de datos, el flujo realiza un *upsert* inicial en la tabla **Contactos** (nodo `Contactos (create: record)`) y posteriormente vincula el identificador resultante al crear la fila en la tabla **Tickets** (nodo `Tickets (create: record)`).

3. Construye el "Corazón" (Orquestación Lógica en n8n)

```
[Webhook POST] ──> [If1: messages exists] ──(true)──> [Edit Fields] ──> [Contactos CRM] ──> [Wait] |
┌──────────────────────────────────────────────────────────────────────────────────────────────┘ ▼ [Basic LLM
Chain] <── [Google Gemini Chat Model] | ▼ [Code in JavaScript] ──> [Tickets CRM] ──> [If: categoria ==
'ESPECIFICA'] | ├──(false: GENERAL)──> [Send message (al cliente)] | └──(true: ESPECIFICA)──> [automensaje (al
cliente)] | ▼ [mensaje especifico (al operador)]
```

3.1. Trigger Inteligente & Filtro Anti-Loop

- **Webhook Productivo:** Configurado en método `POST` sobre la ruta `/webhook/whatsapp` con respuesta inmediata (`Respond: Immediately` con código HTTP `200 OK`) para confirmar la recepción a Meta en menos de 50ms y prevenir reintentos masivos de red.
- **Nodo If1 (Prevención de Bucles Infinitos):** Evalúa rigurosamente: `{{ $json.body.entry[0].changes[0].value.messages }}` **exists**.
  Si Meta envía confirmaciones de entrega (`status: delivered`), lecturas (`status: read`) o ecos de mensajes salientes, el flujo toma la rama `false` (sin nodos conectados) y finaliza en 5 milisegundos, eliminando el consumo innecesario de créditos del LLM y consultas a bases de datos.

3.2. Motor de IA: Basic LLM Chain + Google Gemini Chat Model

Se integra el modelo **Google Gemini Chat Model** mapeado de forma dinámica al contenido del mensaje entrante del cliente. El System Prompt implementado en producción garantiza consistencia terminológica y delimita la salida en JSON estricto sin bloques markdown:

```
// Prompt del Sistema implementado en el nodo Basic LLM Chain
Eres el asistente de atención al cliente de la empresa. Analiza el mensaje recibido:
"{{ $json.mensaje_cliente }}"

Clasifícalo en una de estas dos categorías:
1. "GENERAL": Preguntas frecuentes, saludos, información básica, dudas comunes o precios estándar.
2. "ESPECIFICA": Casos complejos, presupuestos personalizados, quejas, reclamos o cuando el cliente
pide hablar con una persona.

Responde ÚNICAMENTE en formato JSON plano (sin formato markdown ni bloques ```json):
{
  "categoria": "GENERAL",
  "respuesta_cliente": "Texto de la respuesta para el cliente",
  "resumen_para_humano": ""
}

Si la categoría es "ESPECIFICA":
- En "respuesta_cliente" escribe un mensaje amable avisándole al cliente que su consulta fue
transferida a un asesor humano.
- En "resumen_para_humano" escribe un resumen conciso de lo que necesita el cliente.
```

3.3. Gestión de Errores y Resiliencia en JavaScript

El nodo **Code in JavaScript** procesa la carga útil de Gemini, implementando una cláusula `try/catch` como directiva de resiliencia ante eventuales fallas de parsing o respuestas con caracteres inesperados:

```
const raw = $json.text || $json.response || '';
const cleanText = raw.replace(/```json/g, '').replace(/```/g, '').trim();

let parsed = {};
try {
  parsed = JSON.parse(cleanText);
} catch (e) {
  // Directiva de resiliencia / Fallback supervisado
  parsed = {
    categoria: "GENERAL",
    respuesta_cliente: "Hemos recibido tu consulta y estamos procesándola en breve.",
    resumen_para_humano: "Error de parsing en salida LLM. Texto original: " + raw
  };
}

return {
  categoria: parsed.categoria,
  respuesta_cliente: parsed.respuesta_cliente,
  resumen_para_humano: parsed.resumen_para_humano || '',
  telefono_cliente: $('Edit Fields').first().json.telefono_cliente || ''
};
```

4. Implementa el "Human-in-the-Loop"

Para neutralizar el *"Efecto Metralleta"* (emisión de respuestas desalineadas o compromisos comerciales erróneos en casos críticos), el flujo bifurca condicionalmente antes de cualquier resolución definitiva:

- **Detección y Retención:** Si `categoria === 'ESPECIFICA'`, el sistema no formula respuestas comerciales concluyentes de forma autónoma.
- **Secuencia de Validación y Alerta:**
  1. **Aviso de Retención al Cliente (`automensaje`):** Notifica al usuario que su solicitud está en manos de un asesor técnico humano, calmando la ansiedad de espera.

2. **Escalación Prioritaria (`mensaje especifico`):** Envía de inmediato una notificación proactiva al WhatsApp privado del operador corporativo con el teléfono internacional del cliente y el `resumen_para_humano` provisto por la IA.

- **Control de Estado en Airtable:** El registro en la tabla **Tickets** se marca automáticamente como `Escalado a Humano`, requiriendo intervención y resolución directa desde el CRM.

5. Conecta la "Voz" (Salida Multicanal WhatsApp Cloud API)

- **Estandarización E.164:** Todos los nodos de envío (`Send message`, `automensaje` y `mensaje especifico`) utilizan números normalizados bajo el estándar internacional con prefijo de país (ej. `54911xxxxxxxx`).
- **Seguridad de Credenciales:** Se autentica contra la Graph API de Meta mediante un **Token de Acceso Permanente** asociado a un Usuario del Sistema en Meta Business Suite, con permisos de alcance limitado a `whatsapp_business_messaging` y `whatsapp_business_management`.

6. Prueba, Documenta y Entrega

6.1. Matriz de Test de Estrés (Happy & Unhappy Paths)

Caso de Test	Payload de Entrada	Comportamiento Esperado	Resultado Obtenido	Estado
Test 1 (General)	"Hola, ¿cuáles son los horarios de atención y su ubicación?"	Gemini clasifica como `GENERAL`. Salida por rama false hacia `Send message`.	Cliente recibe respuesta en 2.3s; ticket guardado en Airtable.	**PASS**
Test 2 (Específica)	"Requiero presupuesto personalizado para un proyecto de 500 puestos."	Clasificación `ESPECIFICA`. Se dispara `automensaje` y luego `mensaje especifico`.	Cliente recibe aviso; operador recibe alerta con síntesis técnica.	**PASS**
Test 3 (Unhappy: Eco)	Evento de acuse de recibo de Meta (`status: delivered` / `read`).	El nodo `If1` detecta ausencia de `messages` y detiene el flujo en `false`.	Ejecución finalizada en 6ms sin bucle ni consumo de tokens.	**PASS**
Test 4 (Unhappy: Malformación)	Mensaje con caracteres no compatibles o respuesta LLM malformada.	El nodo de código captura el error vía `catch` y aplica respuesta por defecto.	El flujo no colapsa; registra evento seguro en la tabla de tickets.	**PASS**
Test 5 (Handshake GET)	Petición de verificación inicial de Meta con `hub.challenge`.	El endpoint responde código HTTP 200 con el valor numérico exacto en texto plano.	Meta aprueba la URL con tilde verde de producción.	**PASS**

CHECKLIST DE SEGURIDAD Y AUDITORÍA TÉCNICA

✓ **Filtro Anti-Loop:** Nodo `If1` validando `messages exists` para prevenir bucles infinitos con eventos de Meta.

✓ **Tipos de Datos Correctos:** Comparación estricta de string (`categoria == 'ESPECIFICA'`) sin coerción errónea.

✓ **Prompt Dinámico y Seguro:** Inyección acotada de la variable `{{ $json.mensaje_cliente }}` con instrucciones deterministas.

✓ **Protección de Credenciales:** Token permanente protegido en el gestor de credenciales de n8n; claves sensibles ocultadas en la documentación visual.